一、前言
二、软件功能
2.1 可视化工作流系统
2.2 节点控制与数据传输机制
2.3 实验设备抽象与兼容系统
2.4 用户交互与配置功能
2.5 数据分析与实验一体化系统（※灵活交互，独立于工作流）
2.6 系统级支持与跨平台能力
三.程序结构与模板
3.1 程序架构
3.1.1 顶层
3.1.2 UI层
3.1.3 核心逻辑层
3.2 核心对象架构
3.2.1 设备层（Devices）
3.2.2 节点层（Nodes）
3.2.3 工作流层（Workflows）
四、数据结构与对象API规范
4.1 模板与规范
4.1.1 节点模板与API接口规范
4.1.2 设备模板与API接口规范
4.1.3 数据分析面板模板与API规范
4.1.4 工作流类与API模板
4.2 关键数据结构定义
4.2.1 DataPacket：传输数据封装
4.2.2 NodeInfo：节点信息字典
4.2.3 DeviceInfo：设备信息字典
4.2.4 WorkflowInfo：工作流信息字典
4.3 Docstring规范
五、测试环境与测试用例
5.1 测试环境与标准
5.1.1 硬件环境
5.1.2 软件环境
5.1.3 测试标准
5.2 测试用例
5.2.1 CW谱扫描 - 非流式作图
5.2.2 CW谱扫描 - 流式作图
5.2.3 IIR连续测量 - 流式作图
5.2.4 实验参数优化 - 网格化扫描 - 流式作图
5.2.5 实验参数优化 - 启发式搜索 - 流式作图
5.2.6 设备功能控制

5.2.7 偏置磁场优化 - 流式作图
六、部署与分发机制
6.1 Windows平台部署
6.2 Linux平台部署
6.3 分发机制
附录
A. 参考资料

一、前言

本需求文档意图基于python语言实现一个轻量级的实验可视化结点界面，类似ComfyUI，可支持用户级的积木化二次开发。每个实验涉及到一系列可视化的工作流，工作流间有明确的输入、输出可视化排布关系，数据间进行全向量化操作，对不同实验设备具备兼容性。

二、软件功能

本节详细列举系统核心功能模块，涵盖**可视化工作流编辑**、**节点控制机制**、**设备适配**、**数据分析**、**用户交互及系统级部署支持**六大功能。系统目标是提供一套**高度可配置、模块化、面向科研实验场景**的图形化实验设计与执行平台，支持用户快速搭建并运行复杂实验流程。

每一子模块不仅是系统独立组成单元，也支持高度耦合与协作，从而实现“所见即所得”的实验流程构建体验。

模块	功能范围
A. 可视化工作流系统	节点、连接、画布交互、DAG图执行
B. 节点控制与数据传输机制	节点分类、接口类型、数据结构、向量化
C. 实验设备抽象与兼容系统	设备适配、模拟、统一接口
D. 用户交互与配置功能	UI 操作、变量、日志、异常、快捷操作
E. 数据分析与实验一体化系统	数据导入、报告生成、样式复用
F. 系统级支持与跨平台能力	打包、环境配置、平台兼容性

2.1 可视化工作流系统

功能	描述
节点式工作流构建	支持拖拽节点控件、连线、参数配置形成工作流
工作流画布布局优化	支持自动布局（层级顺序布局）提升可读性
DAG图执行机制	自动解析节点依赖顺序，保证拓扑结构正确，根据DAG图关系自动进行拓扑排序层级
左→右自然顺序执行	节点从左向右执行，与逻辑流一致
并行子图执行	若多个子图无依赖关系，则使用多线程并发处理
节点执行就绪验证	所有输入满足后节点才能执行，避免空输入错误
撤销 / 回撤 / 重做	工作流画布支持 Ctrl+Z / Ctrl+Shift+Z 等标准操作
复制 / 粘贴节点	支持节点、子图级复制与粘贴
工作流节点化复用	可将完整工作流封装为“复合节点”进行嵌套调用
分页式工作流管理	用户可同时打开多个工作流页面，每个页面具备独立变量空间、节点图与设备状态
工作流库管理	支持命名、保存、载入、分组管理工作流图结构
状态高亮与运行指示	正在运行节点高亮显示
标签可重命名	支持对工作流页签重命名、右键菜单另存为/关闭

2.2 节点控制与数据传输机制

功能	描述
节点类型定义	支持 6 大类型：输入输出、设备配置、数据读取、数据处理、数据可视化、控制流
节点接口分六种类型	输入、流输入、控制输入、输出、流输出、控制输出
节点接口数据标准化	所有接口传输的数据采用统一 dict格式，便于校验和转化、隔离数据格式与内容异常
节点自动识别与挂载	自动扫描nodes/、devices/等本地目录并挂载到库中进行显示调用

功能	描述
全流程向量化数据结构	所有节点都以 NumPy 向量或等效结构传输数据，支持高维数据
循环嵌套自动支持	控制流节点（如循环）支持嵌套子图并根据循环嵌套自动生成向量化结构
缓存与维度压缩	防止流式采集/大范围扫描导致内存溢出，可配置自动分段保存与数据维度压缩（统计向量化操作）
属性编辑器配置节点	节点点击后加载对应参数编辑界面，动态绑定字段变化
数据可视化支持	数据输出可整段/流式送入作图节点，作图由plot_manager管理并集中展示
输出接口具备 shape 信息	用于数据维度检查与兼容性预判
节点数据持久驻留	执行完的节点保留数据，可反复分析与导出使用

2.3 实验设备抽象与兼容系统

功能	描述
设备统一管理器	支持多设备注册、基于硬件端口号和序列号进行设备连接、通讯测试、配置、内存中状态跟踪等功能
标准化设备接口类	每类设备基于统一 DeviceBase派生，标准化控制/读数方法的基本入口
支持多实例同类设备	可同时载入多个同型号设备，分配唯一ID与参数空间
工作流间设备连接策略	若设备资源唯一（如USB设备），应实现设备使用锁或自动互斥策略，避免多个工作流间存在重叠，或在单个工作流运行结束后释放资源
用户定义设备参数结构	每台设备配置面板可加载自定义的参数格式模板
仿真设备类	每类真实设备均有同构仿真类，方便无硬件调试开发
驱动与资源独立封装	所有硬件相关依赖集中于 resources/drivers/
热插拔机制（可扩展）	支持动态连接/释放设备资源，便于故障恢复和调试

2.4 用户交互与配置功能

功能	描述
全局变量管理器	用户可定义符号变量并引用于节点参数中，或指定节点输出为符号变量
符号变量自动提取	自动扫描/实时监测工作流中节点，提取符号定义生成符号表
参数内符号调用	参数字段中支持如 ${var}的变量引用与实时预览值
实时状态反馈	当前执行中的节点/流式传输连接线高亮、状态栏显示当前节点名称与状态
全局事件信号系统	所有重要事件统一发送信号，如节点生成、数据传输、执行状态变更等
多级日志系统	支持 info / warning / error 等日志等级，UI中分色显示，自动保存日志文件到本地，便于故障追溯
异常捕获机制	节点级 try/except 包裹，支持中断当前节点/流程，记录 Traceback
快捷键支持	Ctrl+S 保存，F5 执行，Ctrl+C/V 复制粘贴，F1 打开帮助等常规操作提示，每页快捷键需独立绑定工作流，避免影响非当前页
操作提示与工具提示	悬浮提示、错误反馈、运行提示增强用户体验

2.5 数据分析与实验一体化系统（※灵活交互，独立于工作流）

模块	描述
图形化数据分析面板	ui/analysis_panel.py总控，托管各类子面板
分页式分析面板体系	每个子模块（如 CW 拟合）在 ui/analysis/下独立维护
本地数据导入分析	支持加载已有实验数据（CSV、JSON、npy）进行处理
节点数据反向提取分析（升级目标）	通过 core/data_router.py与节点结果对接，提取特定节点输出供分析使用
分析图表导出保存与重用	图表、拟合参数可保存为图片或 JSON 分析模板

模块	描述
分析结构支持插件化扩展	支持用户开发自定义分析模块并注册到分析面板系统中
可视化组件共用	可部分复用 plot_widget.py/ plot_manager.py实现高质量分析绘图

2.6 系统级支持与跨平台能力

功能	描述
跨平台兼容	支持 Windows / Linux 系统，确保在 Raspberry Pi 4B 上可流畅运行
运行性能优化	UI 性能测试基准包含内存/CPU占用控制、节点渲染优化等
虚拟环境打包	配置 requirements.txt + venv自动部署工具
一键系统配置脚本	含 PySide6 环境、驱动库、设备依赖、系统权限配置等脚本（如 bash / bat）
日志与配置持久化	所有配置与日志可持久保存，便于故障恢复和实验复现
示例与模板支持	workflows/与 devices/中包含多个预设样例与仿真示例
文档自动生成	基于 docstring自动生成 API 参考文档到 docs/

三.程序结构与模板

3.1 程序架构

```
experiment_flow_ui/
├── main.py # 程序入口点
├── readme.md # 程序文档
├── requirements.txt # 设备依赖
├── setup/ # 安装打包文件夹
│   ├── setup_windows.bat # Windows平台自动安装程序
│   ├── setup_linux.sh # Linux平台自动安装程序
│   ├── windows_runtime.zip # Windows平台运行库等依赖
│   ├── linux_runtime.zip # Linux平台运行库等依赖
│   └── pyenv.zip # Python虚拟环境
```

```
├── config/ # 配置文件夹
│   ├── system_config.ini  # 系统配置
│   ├── device_config.ini  # 设备配置，兼容不同工作流
│   └── config_loader.py  # 配置解析器
├── ui/ #
│   ├── analysis/    # 嵌入数据分析面板的子面板空间，notebook分页，用户可独立开发
│   │   ├──cw_spectrum_fit_panel.py # CW谱分析拟合面板
│   │   ├──params_optimization_panel.py # 参数优化结果分析面板
│   │   └── ...
│   ├── main_window.py    # 主窗口
│   ├── workflow_library.py   # 本地工作流
│   ├── device_library.py   # 本地设备库
│   ├── canvas.py         # 基于QGraphicsScene的绘图逻辑
│   ├── workflow_tab_panel.py   # 多个工作流分页容器（QTabWidget）
│   ├── single_workflow_page.py  # 单个工作流页面（封装画布 + 属性栏 + 状态栏 + 变量面板）
│   ├── node_library.py         # 节点库
│   ├── node_widget.py    # Node控件
│   ├── edge_widget.py    # 连接线控件
│   ├── property_panel.py # 属性编辑器
│   ├── device_manager.py # 设备管理器
│   ├── variable_manager.py # 用户定义符号变量管理器
│   ├── plot_manager.py         # 数据可视化作图管理器
│   ├── plot_widget.py         # 单张可视化作图控件
│   ├── log_panel.py # 日志面板
│   ├── status_bar.py # 状态面板，实时显示实验进度、CPU/内存资源占用
│   ├── analysis_panel.py # 数据分析面板，实验数据生成/数据处理界面一体化
│   └── menu_bar.py # 菜单栏，包括文件、选项、帮助等选项卡
├── core/ # 核心代码逻辑
│   ├── graph_executor.py   # 工作流DAG解析与执行
│   ├── base_workflow.py      # WorkFlow基础类，包含接口检查、生成
│   ├── base_node.py        # Node基础类
│   ├── base_device.py      # Device基础类
│   ├── registry.py         # 注册机制
│   ├── execution_context.py    # 单次实验信息封装
│   ├── exception_handler.py   # 异常管理机制
│   ├── data_router.py   # 数据分析和工作流间数据传输机制 （※首先实现本地功能）
│   └── variables.py        # 符号变量管理机制
├── nodes/ # 节点库模板，用户可自定义扩展
```

```
49  │   ├── __init__.py # 注册
50  │   ├── input_output.py # 工作流控件输入、输出节点
51  │   ├── configure_device.py # 设备参数配置节点
52  │   ├── data_read.py # 数据读取节点
53  │   ├── data_process.py # 数据处理节点
54  │   ├── visualization.py # 可视化节点
55  │   ├── debug.py # 调试节点，支持断点、打印输出值等操作
56  │   ├── composite_workflow.py # 嵌套工作流，封装子工作流为节点
57  │   └── control_flow.py # 控制流控件，循环、条件判断、延时等操作
58  ├── devices/ # 设备库，用户可自定义扩展
59  │   ├── __init__.py # 设备统一注册
60  │   ├── device_simulators.py # 设备基类
61  │   ├── lockin_amplifier/ # 锁相放大器
62  │   │   ├── __init__.py # 注册
63  │   │   ├── lockin_amplifier_simulator.py # 锁相放大器仿真类
64  │   │   └── LIA_Mini_V3_USB_2CH.py # 示例，科大小型化锁相，2通道固件版本
65  │   ├── microwave/ # 微波源
66  │   │   ├── __init__.py # 注册
67  │   │   ├── microwave_simulator.py # 微波源仿真类
68  │   │   └── CYQTEK_MW_2CH.py # 示例，国仪微波源，2通道版本
69  │   ├── laser/ # 激光器
70  │   │   ├── __init__.py # 注册
71  │   │   ├── laser_simulator.py # 激光器仿真类
72  │   │   └── CYQTEK_CSS_V5.py # 示例，国仪电流源
73  │   ├── motor/ # 电机
74  │   │   ├── __init__.py # 注册
75  │   │   ├── motor_simulator.py # 激光器仿真类
76  │   │   └── usm20.py # 示例，usm20电机驱动器
77  │   └── ...
78  ├── resources/ # 资源
79  │   ├── drivers/ # 驱动与运行库资源
80  │   ├── icons/ # 图像资源
81  │   └── themes/ # 可视化风格配置
82  ├── workflows/ # 工作流示例文件
83  │   ├── cw_scan_odmr.json # 扫描CW谱实验
84  │   └── ...
85  ├── docs/ # 基于docstring自动生成格式化文档
86  └── utils/ # 工具
87      ├── file_io.py # 工作流、配置文件、实验数据的统一读写
88      └── logger.py # 全局日志管理器，支持多等级、UI输出、文件持久化的日志模块
```

3.1.1 顶层

路径	职责说明
main.py	应用程序入口，启动 UI、载入配置、初始化工作流环境
readme.md	项目文档说明（详细描述模块功能、依赖与示例）
requirements.txt	设备依赖说明
config/	配置分离，含解析器模块
workflows/	本地保存的工作流图 JSON 文件，建议每个文件都具备 UUID 和版本管理字段

3.1.2 UI层

模块	职责
main_window.py	主窗口框架，集成菜单栏、工作流画布、属性面板、状态栏
canvas.py	绘图主区域，基于 QGraphicsScene处理节点增删、拖放、连线、布局、缩放显示等操作
node_widget.py	节点 UI 组件（与 base_node.py 配合构建）
edge_widget.py	节点连线组件，支持动态连线、样条曲线spline美化、双向高亮等
property_panel.py	显示并修改当前选中节点属性（根据节点属性/设备属性）
workflow_library.py	管理本地保存的工作流集合，可视化、载入、另存为等功能（支持搜索、刷新显示）
device_library.py	管理本地保存的设备实例集合（支持搜索、刷新显示）
device_manager.py	设备管理器，可对某一设备进行详细检查和单独控制
node_library.py	展示所有支持的节点模板（点击/拖拽添加）
workflow_tab_panel.py	多个工作流分页容器
single_workflow_page.py	单个工作流页面，具备完整的画布、属性、状态、参数名称空间，全部单独实例化
variable_manager.py	管理用户定义变量，支持基础公式解析、符号替换
plot_manager.py	控制绘图逻辑，包括窗口管理、多图更新、实时作图更新、滚动显示配置、统计功能、自动缩放
plot_widget.py	实际绘图控件，基于 pyqtgraph实现，支持不同作图类型，轻量化加速
analysis_panel.py	数据处理面板，支持用户通过图形界面现场处理数据，将数据分析软件和实验工作流一体化

模块	职责
log_panel.py	用户自定义全局日志，支持等级过滤、按工作流过滤、自动滚动、保存到本地
menu_bar.py	菜单栏控制器，挂载文件、帮助、配置等选项卡
status_bar.py	状态栏模块，显示当前节点、线程、进度信息，每页单独实例化

3.1.3 核心逻辑层

模块	职责
graph_executor.py	解析 DAG，调度节点执行；控制执行状态、异常处理与数据传递
base_workflow.py	工作流模型，包含子图、自动序列化、接口定义等功能
base_node.py	所有节点的父类，定义输入/输出接口，执行流程，UI绑定接口等
base_device.py	所有实验设备的父类，定义设备初始化、参数设置、数据读取等接口
registry.py	节点类型、设备类型、执行模板等统一注册机制，支持动态扩展
exception_handler.py	定义管理统一异常类型（如 NodeExecutionError、DeviceError、GraphTopologyError）
execution_context.py	封装一次实验的上下文，包括变量表、设备连接池、运行日志、状态记录。
variables.py	全局变量识别与管理（考虑配合 sympy进行符号计算？）

3.2 核心对象架构

该层级描述系统中三个核心对象的结构与组织方式，是系统功能运行的基础支撑。

3.2.1 设备层（Devices）

- 所有实验设备封装为独立Python类，继承自 DeviceBase，支持初始化、参数配置、命令下发与数据读取。
- 同类设备可多实例化，每个实例具备唯一标识符与隔离状态空间。
- 支持物理设备与仿真类解耦开发，开发者可在无硬件条件下调试核心功能。
- 所有设备以模块化形式注册至 devices/ 目录，统一通过 registry.py 管理。

3.2.2 节点层（Nodes）

- 节点为系统中最基本的计算单元，封装了参数定义、执行逻辑与输入输出接口。
- 节点分为六类：输入输出、设备配置、数据读取、数据处理、数据可视化、控制流。
- 所有节点继承自 BaseNode，以标准接口进行连接与调度，支持复合节点嵌套。
- 节点定义存放于 nodes/，支持用户添加自定义节点并通过自动扫描加载。

3.2.3 工作流层（Workflows）

- 工作流为多个节点组合而成的图结构，具有有向无环图（DAG）拓扑属性。
- 每个工作流由 BaseWorkflow 实例表示，支持序列化保存、变量隔离、多页并行运行。
- 工作流支持嵌套复用、分组管理与页面资源绑定，可作为节点重复调用。

四、数据结构与对象API规范

4.1 模板与规范

4.1.1 节点模板与API接口规范

4.1.2 设备模板与API接口规范

4.1.3 数据分析面板模板与API规范

4.1.4 工作流类与API模板

4.2 关键数据结构定义

4.2.1 DataPacket：传输数据封装

4.2.2 NodeInfo：节点信息字典

4.2.3 DeviceInfo：设备信息字典

4.2.4 WorkflowInfo：工作流信息字典

4.3 Docstring规范

代码中类、公开函数的docstring统一采用 **reStructredText** 风格规范。

五、测试环境与测试用例

本节旨在通过典型实验工作流配置示例，说明系统在不同科研实验任务中的功能构建方式，涵盖从基础设备配置、数据采集、处理、可视化到参数优化等多类使用场景。各示例可作为用户上手指南、系统验证基准或节点模板复用参考。

5.1 测试环境与标准

5.1.1 硬件环境

- Windows 10/11 (x64)，Raspberry Pi 4B (4GB+)
- 实验设备包括锁相放大器、激光控制器、微波源、电机控制器，含真实设备与模拟类

5.1.2 软件环境

- Python 3.10+
- 依赖：PySide6、numpy、pyqtgraph、pyserial、paramiko 等
- 统一基于 requirements.txt 或封装好的setup/pyenv.zip进行虚拟环境配置

5.1.3 测试标准

- **UI流畅性**：100+节点的复杂图无拓扑异常，执行顺序正确，渲染无明显卡顿，节点拖拽无明显卡顿，数据刷新帧率 ≥ 20 fps，可视化数据无显著延时、卡顿
- **数据完整性**：1kHz采样率节点间数据传输无丢失，各类接口校验功能正常工作
- **数据采集流畅性**：5Msps@0.1s、25Msps@0.01s、100kHz@1s采集深度的DAQ模式连续1000组采集不丢点
- **连续采集**：含图形界面的连续运行程序稳定工作 ≥ 144小时，无显著延时、丢点、内存泄漏
- **工作流规划执行正确性**：100+节点的复杂图无拓扑异常，所有规定的测试用例上执行顺序和输出结果正确

5.2 测试用例

5.2.1 CW谱扫描 - 非流式作图

※**场景描述**：该示例展示一个最基础的 CW（连续波）光谱扫描流程，不包含实时数据刷新，仅在循环完成后进行统一的数据可视化与保存。适用于软件演示、设备测试、静态分析等任务。

（1）参数配置：锁相放大器

- 拖入“参数配置”节点，可重命名为 参数配置：锁相放大器配置
- 设备类型：锁相放大器
- 配置参数：调制频率、采样频率等（数值需通过类型校验）

（2）微波扫频循环

- 拖入“循环”节点，可重命名为 循环：微波扫频
- 定义循环变量 mw_freq
    - 起始频率 start_mw_freq
    - 终止频率 stop_mw_freq
    - 点数 scan_mw_points
- 输出端口自动生成：mw_freq

（3）设置微波源参数

- 拖入“参数配置”节点，可重命名为 参数配置：微波设置
- 放置为“循环：微波扫频”子节点
- 设备类型：微波源
- 参数绑定：将 mw_freq 连接至该节点的“微波中心频率”输入口（数值需通过类型校验，否则抛出错误）

（4）读数操作：锁相输出

- 拖入“读取数据”节点，可重命名为 读取数据：锁相扫谱
- 放置在微波设置节点之后，确保顺序执行
- 设备类型：锁相放大器
- 参数配置：
  - 配置采集通道：input1-x, input1-y
  - 配置数据点数：cw_scan_pts

（5）循环输出数据结构

- 循环：微波扫频 节点根据子节点数据自动生成输出接口 cw_data
- 数据维度：(scan_mw_points, 2, cw_scan_pts)，其中 2 为两个通道

（6）数据处理：平均

- 拖入“数据处理”节点，可重命名为 数据处理：平均
- 功能：对 cw_data 第三维度求平均，降维为 (scan_mw_points, 2)
- 输出接口命名：cw_data_avg

（7）数据可视化

- 拖入“数据可视化”节点，可重命名为 数据可视化：CW谱
- 输入连接：数据处理：平均 的 cw_data_avg
- 作图方式：折线图，作图要素（图例、标签、）由用户定义，横轴为频率变量 mw_freq

（8）数据保存

- 拖入“数据保存”节点，可重命名为 数据保存：CW谱
- 连接数据源：cw_data_avg
- 配置保存路径、格式（如 .csv, .npy）

（9）实验执行与完成

- 点击“开始实验”
- 系统依据 DAG 拓扑自动执行节点任务
- 当前执行节点高亮显示，错误信息由日志面板提示
- 实验完成后，所有数据驻留在对应节点中供后续分析

5.2.2 CW谱扫描 - 流式作图

※**场景描述**：CW谱实验的进阶配置，支持在循环过程中实时刷新作图画布。用于现场监控谱图动态变化，提升实验调试效率。用于演示完整的流式作图功能，和现场实验条件一致。

关键配置：

与 1.2.1 的结构相似，差异如下：

- “数据可视化”节点改为“流式显示”模式
- 添加斜率与线宽拟合节点（可选）
- 若设备支持同步触发，可启用触发同步机制提高刷新一致性

5.2.3 IIR连续测量 - 流式作图

※**场景描述**：长时间噪声测量场景，验证软件在大批量、高频率采样条件下的数据吞吐性能与流式绘图稳定性。

关键配置：

- 流式“读取数据”节点设置采样时间窗口
- 接收端“流数据可视化”节点启用缓存区自动保存机制
- 使用滑窗平均或降采样节点减缓绘图压力

5.2.4 实验参数优化 - 网格化扫描 - 流式作图

※**场景描述**：在二维参数空间中遍历所有组合，获取目标函数（如信号强度）并绘制热图，用于演示基础多层嵌套功能和数据矢量化操作。

关键配置：

- 使用双层“循环节点”：
    - 外层变量 param_a

    - 内层变量 param_b
- 节点输出为多个三维数组 (param_a_points, param_b_points, data_dim)
- 数据处理节点对末维进行统计
- 使用“热图作图节点”绘制二维参数热图

5.2.5 实验参数优化 - 启发式搜索 - 流式作图

※**场景描述**：启发式算法（如梯度上升、遗传算法）驱动参数优化，动态生成采样路径并实时绘制性能图谱。，用于演示循环嵌套结构、条件判断和热图可视化功能。

关键配置：

- 控制流节点内嵌判断、赋值、变量更新模块
- 热图节点支持动态更新区域与路径标记

5.2.6 设备功能控制

※**场景描述**：单个设备功能控制的实施例，用于演示验证对单个设备的配置下发能力/数据读取能力

关键配置：

- 适合设备驱动开发完成后的功能验证
- 每个设备类型建议具备一个对应的基础功能测试工作流模板

5.2.7 偏置磁场优化 - 流式作图

※**场景描述：**本例为演示基于直流荧光分量快速条件偏置磁场的实施例，需要以并行线程形式连续采集超声电机角度数据和直流荧光数据

关键配置：

- 数据来源：激光荧光信号（dc）、角度传感器读数
- 多线程“数据读取节点”异步读取两类信号
- 控制流节点持续执行，直到满足信号收敛条件

六、部署与分发机制

6.1 Windows平台部署

- 建议使用系统Windows 11
- 安装Python环境
- 使用一键安装脚本 setup_windows.bat 自动创建虚拟环境、配置环境变量、解压并安装所需库
- 启动入口：在虚拟环境终端中运行 python main.py，或封装为main.exe一键运行

6.2 Linux平台部署

- 建议使用系统Raspberry Pi OS (Debian-based)
- 使用 一键安装脚本setup_linux.sh 完成依赖安装与权限配置
- 部署到工程样机上时，可通过Terminal从命令行自动加载运行特定工作流
- 部署到工程样机上时，需要做好必要的反编译工作

6.3 分发机制

支持以下形式分发：

- 打包工作流模板、设备配置与软件安装脚本为 zip 压缩文件，解压后遵循readme.md进行安装
- 用户可通过导入 .json 工作流图与 .ini 设备配置文件复现完整实验

附录

A. 参考资料

【1】https://github.com/comfyanonymous/ComfyUI ComfyUI开源项目

【2】 https://docs.zhinst.com/mfli_user_manual/functional_description/labone_overview.html Labone设备UI界面

【3】 https://www.ni.com/en/shop/labview/select-edition/labview-community-edition.html Labview实验控制软件社区版

【4】https://en.wikipedia.org/wiki/ReStructuredText reStructredText接口文档规范